Prueba de Desempeño .NET — SkillHub

Contexto

SkillHub es un centro de formación que ofrece talleres cortos y presenciales de fotografía, cocina, oratoria, diseño y otras áreas.
Hoy las inscripciones se reciben por un formulario y se controlan en hojas de cálculo. Esto genera talleres con más inscritos que cupos, personas inscritas dos veces en el mismo taller y participantes inscritos en dos talleres a la misma hora.
SkillHub quiere una plataforma donde los participantes consulten los talleres y se inscriban, y donde los empleados del centro publiquen talleres y controlen quién asiste a cada uno.

Objetivo

Construir una solución en .NET compuesta por una **API REST** y una **capa de presentación** que permita a SkillHub gestionar sus talleres e inscripciones.
Esta prueba evalúa tu capacidad para:

- Diseñar una API aplicando Clean Architecture.
- Modelar reglas de negocio reales en el dominio, más allá de un CRUD.
- Proteger la información con autenticación y autorización.
- Consumir esa lógica desde una aplicación cliente.

Todo lo que has aprendido en la ruta es válido y esperado en esta prueba.

Requerimientos funcionales

Usuarios y acceso

- Cualquier persona puede registrarse en la plataforma. Quien se registra lo hace siempre como **participante**.
- Los empleados de SkillHub son sus instructores y ya cuentan con una cuenta, con la que se debe poder ingresar desde el primer arranque de la aplicación (No olvides dejar el acceso en el readme).
- Los empleados del centro publican talleres y consultan los inscritos de cualquier taller.
- Un participante solo puede ver y gestionar **sus propias** inscripciones. Nunca las de otro participante.
- Cualquier visitante, aunque no haya iniciado sesión, puede consultar los talleres disponibles.

Instructores

De cada instructor se necesita conocer como mínimo: nombre completo, correo y especialidad. Todo taller es dictado por un instructor registrado en la plataforma.

Talleres

De cada taller se necesita conocer como mínimo: código único (por ejemplo, FOT-001), título, descripción, el instructor que lo dicta, fecha y hora de inicio, fecha y hora de fin y cupo máximo.

- Solo los empleados pueden crear talleres. Al crearlo, el empleado indica el instructor por su correo y este debe existir en la plataforma.
- Un taller se crea siempre con fecha futura y la hora de fin debe ser posterior a la de inicio.

- El catálogo muestra solo los talleres que aún no han comenzado, junto con sus **cupos disponibles**. Los cupos disponibles los calcula el sistema.

Inscripciones

Un participante se inscribe en un taller. Toda inscripción nace **Activa** y puede pasar a **Cancelada**.

1. **Cupo.** No se puede superar el cupo máximo del taller. Solo cuentan las inscripciones activas.
2. **Sin duplicados.** Un participante no puede tener dos inscripciones activas en el mismo taller.
3. **Sin cruces de horario.** Un participante no puede inscribirse en un taller que se cruce en horario con otro donde ya tiene una inscripción activa.
4. **Solo talleres futuros.** No es posible inscribirse en un taller que ya comenzó.
5. **Cancelación.** El participante puede cancelar su inscripción hasta **24 horas** antes del inicio del taller. Al cancelar se libera el cupo. Un empleado también puede cancelar la inscripción de cualquier participante sin la restricción de 24 horas, siempre que el taller no haya comenzado. Toda cancelación debe registrar quién la realizó y cuándo.
6. **Reinscripción.** Un participante que canceló puede volver a inscribirse en el mismo taller si aún hay cupo y se cumplen las demás reglas.

Inscripción masiva desde Excel

Los empleados reciben de otras áreas listados de personas que deben quedar inscritas en talleres. Para no inscribirlas una por una, un empleado puede cargar un archivo Excel donde cada fila indica el correo del participante (`ParticipantEmail`) y el código del taller (`WorkshopCode`).

- Cada fila se procesa como una inscripción normal y debe cumplir todas las reglas anteriores.
- Si el participante o el taller no existen, la fila se rechaza.
- Las filas válidas se guardan aunque otras fallen.
- La respuesta indica cuántas filas se inscribieron y, de cada fila rechazada, su número y el motivo.

Constancia de inscripción

Un participante necesita demostrar que está inscrito en un taller. Para eso puede descargar una **constancia de inscripción en PDF**.

- Solo se genera para inscripciones activas. Una inscripción cancelada no tiene constancia.
- Un participante solo puede descargar la constancia de sus propias inscripciones. Un empleado puede descargar la de cualquier inscripción.
- La constancia incluye como mínimo: nombre de SkillHub, título "Constancia de inscripción", nombre y correo del participante, código, título, instructor, fecha y horario del taller, fecha de inscripción y fecha de generación del documento.
- La librería para generar el PDF es de libre elección. La generación debe quedar en la capa de infraestructura, detrás de una abstracción definida en la capa de aplicación.

Reporte

Los empleados necesitan consultar los **talleres con más inscritos**, ordenados de mayor a menor número de inscripciones activas.

Endpoints de la API

La API debe exponer como mínimo los siguientes endpoints. Puedes agregar los que consideres necesarios. Los endpoints que no son públicos deben exigir un token JWT válido.

Método	Ruta	Acceso	Descripción
POST	`/api/v1/auth/register`	Público	Registra un nuevo participante
POST	`/api/v1/auth/login`	Público	Inicia sesión y devuelve el token JWT
GET	`/api/v1/workshops`	Público	Lista los talleres futuros con sus cupos disponibles
GET	`/api/v1/workshops/{id}`	Público	Detalle de un taller con sus cupos disponibles
POST	`/api/v1/workshops`	Empleado	Crea un taller
GET	`/api/v1/workshops/{id}/enrollments`	Empleado	Lista los inscritos de un taller
POST	`/api/v1/enrollments`	Participante	Inscribe al participante autenticado en un taller
GET	`/api/v1/enrollments/mine`	Participante	Lista las inscripciones del participante autenticado
PATCH	`/api/v1/enrollments/{id}/cancel`	Dueño de la inscripción o empleado	Cancela una inscripción
POST	`/api/v1/enrollments/import`	Empleado	Inscribe participantes en talleres desde un archivo Excel
GET	`/api/v1/enrollments/{id}/certificate`	Dueño de la inscripción o empleado	Descarga la constancia de inscripción en PDF
GET	`/api/v1/reports/top-workshops`	Empleado	Talleres con más inscritos

Las reglas de negocio se validan en el dominio y en la capa de aplicación, no en los controladores. Ante un error, la API debe responder con el código HTTP adecuado y un mensaje claro.

Capa de presentación

Además de la API, debes construir una aplicación para los participantes de SkillHub. Elige **una** de estas dos opciones:

Opción	Qué construir	Cómo se conecta
A. Web con ASP.NET Core	Una aplicación web dentro de la misma solución, como otra capa de presentación	Consume los **servicios de la capa de aplicación**, no los endpoints de la API
B. Cliente frontend	Una aplicación en la tecnología frontend que prefieras	Consume los endpoints de la API usando el token JWT

La aplicación debe cubrir el flujo del participante:

- ☐ Registro e inicio de sesión.
- ☐ Consulta de los talleres disponibles y sus cupos.
- ☐ Inscripción en un taller.
- ☐ Consulta de sus inscripciones.
- ☐ Cancelación de una inscripción.
- ☐ Descarga de la constancia de inscripción en PDF.

Las operaciones de los empleados se demuestran directamente sobre la API, con Swagger, Scalar o Postman.

Requerimientos técnicos obligatorios

Si alguno de estos puntos no se cumple, la prueba se considera incompleta.

1. **Clean Architecture.** La solución debe separar como mínimo Domain, Application, Infrastructure y la capa de presentación (API y, si aplica, la web). Las dependencias apuntan hacia el dominio.
2. **Base de datos con Code First.** El modelo se define en código con Entity Framework Core y la base de datos se genera mediante migraciones.
3. **ASP.NET Core Identity** para la gestión de usuarios.
4. **JWT.** Los endpoints que no son públicos deben estar protegidos con token JWT.
5. **Autorización.** Cada operación debe estar restringida a quien corresponde según los requerimientos funcionales.
6. **Reglas de negocio** implementadas y validadas fuera de los controladores.
7. **Patrones de diseño.** Aplica los patrones vistos en la ruta, como la inyección de dependencias con el contenedor nativo de .NET (las clases dependen de interfaces, no de implementaciones concretas), el patrón Repository y el uso de DTOs...
8. **Capa de presentación** según la opción elegida.
9. **Código en inglés:** clases, propiedades, métodos, carpetas y proyectos.
10. **README en inglés** que incluya:
    - Descripción del proyecto.
    - Tecnologías utilizadas.
    - Pasos para configurar y ejecutar la solución, incluida la base de datos.
    - Credenciales de la cuenta de empleado.
    - Sección **Author** con tu nombre completo, clan, correo y documento de identidad.
11. **Repositorio en GitHub** con el historial de commits del desarrollo hechos durante la prueba.

Recursos

Se entrega el archivo **SkillHub_Data.xlsx** con tres hojas:

Hoja	Contenido	Uso
Instructors	Instructores (empleados) de SkillHub	Deben existir en la base de datos al arrancar la aplicación
Participants	Participantes ya registrados en SkillHub	Deben existir en la base de datos al arrancar la aplicación
Workshops	Talleres ya publicados	Deben existir en la base de datos al arrancar la aplicación
Enrollments	Inscripciones pendientes por cargar	Es la hoja que procesa `POST /api/v1/enrollments/import`

Una vez construida la base de datos, debes cargar en ella los datos de las hojas Instructors, Participants y Workshops; la forma de hacerlo queda a tu criterio. La hoja Enrollments se carga únicamente mediante el endpoint de importación.

Opcionales

Hazlos **solo si terminaste todo lo obligatorio**. Un opcional no compensa ni subsana un requerimiento obligatorio que esté incompleto o mal implementado.

- [ ] Dockerizar el proyecto o los proyectos.
- [ ] Pruebas unitarias.
- [ ] Despliegue de la solución completa.

Entregables

- [ ] El proyecto completo, comprimido, según las indicaciones de tu Team Leader.
- [ ] El enlace al repositorio público en GitHub.
- [ ] El README en inglés en la raíz del repositorio.
- [ ] El script SQL de la base de datos final, con estructura y datos, exportado desde tu gestor de base de datos.